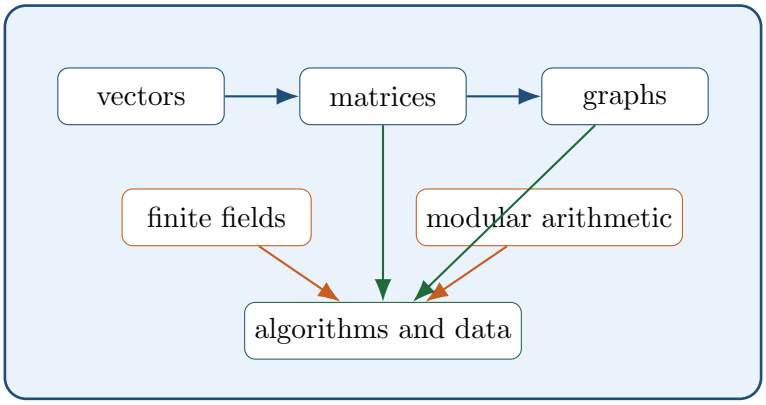
# Algebra for Computer Scientists

Lecture Notes




A mathematical introduction to linear algebra, discrete algebra, and algebraic structures used in computer science.

May 16, 2026

# Contents


**1 Purpose and Scope** **5**

1.1 Prerequisites . . . 5

1.2 Core themes . . . 5

**2 Sets, Relations, and Algebraic Language** **5**

2.1 Sets and operations . . . 5

2.2 Relations and equivalence . . . 6

2.3 Functions and composition . . . 6

**3 Integers, Divisibility, and Modular Arithmetic** **7**

3.1 Divisibility . . . 7

3.2 Euclidean algorithm . . . 7

3.3 Congruence modulo $n$ . . . 7

3.4 Arithmetic in $\mathbb{Z}_n$ . . . 8

3.5 Modular inverse . . . 8

**4 Groups: Symmetry and Reversible Computation** **9**

4.1 Definition of a group . . . 9

4.2 Examples . . . 9

4.3 Permutations . . . 9

4.4 Subgroups . . . 10

4.5 Cyclic groups . . . 10

**5 Rings and Fields** **10**

5.1 Rings . . . 11

5.2 Fields . . . 11

5.3 Why fields matter . . . 11

**6 Polynomials and Finite Fields** **11**

6.1 Polynomial rings . . . 11

6.2 Division algorithm for polynomials . . . 12

6.3 Finite fields via polynomials . . . 12

**7 Vectors and Vector Spaces** **12**

7.1 Vectors as data . . . 12

7.2 Linear combinations and span . . . 13

7.3 Linear independence . . . 13
7.4 Basis and dimension . . . 14

**8 Matrices and Linear Transformations** **14**
8.1 Matrices . . . 14
8.2 Linear transformations . . . 14
8.3 Geometric examples . . . 15
8.4 Matrix multiplication as composition . . . 15

**9 Systems of Linear Equations** **15**
9.1 Matrix form . . . 15
9.2 Gaussian elimination . . . 16
9.3 Rank . . . 16
9.4 Computational complexity . . . 16

**10 Determinants and Invertibility** **17**
10.1 Meaning of determinant . . . 17
10.2 Invertibility . . . 17
10.3 Conditioning . . . 17

**11 Inner Products and Orthogonality** **18**
11.1 Dot product . . . 18
11.2 Orthogonality . . . 18
11.3 Projection . . . 18
11.4 Orthogonal matrices . . . 19

**12 Eigenvalues, Eigenvectors, and Diagonalization** **19**
12.1 Eigenvectors . . . 19
12.2 Characteristic polynomial . . . 19
12.3 Diagonalization . . . 20

**13 Graphs and Matrices** **20**
13.1 Adjacency matrices . . . 20
13.2 Walks and powers of adjacency matrices . . . 20
13.3 Graph Laplacian . . . 20

**14 Boolean Algebra and Logic** **21**
14.1 Boolean values . . . 21
14.2 Boolean algebra as algebra . . . 21

14.3 Boolean rings . . . 21

**15 Algebra over $\mathbb{Z}_2$: Bits, Codes, and Parity** **21**

15.1 Vector spaces over $\mathbb{Z}_2$ . . . 21

15.2 Parity checks . . . 22

15.3 Linear codes . . . 22

**16 Affine Geometry and Homogeneous Coordinates** **22**

16.1 Affine maps . . . 22

16.2 Homogeneous coordinates . . . 22

**17 Recurrences, Matrices, and Algorithms** **23**

17.1 Linear recurrences . . . 23

17.2 Fast exponentiation . . . 23

**18 Semirings and Shortest Paths** **23**

18.1 Why semirings? . . . 24

18.2 Examples . . . 24

**19 Algebraic Thinking in Databases and Formal Languages** **24**

19.1 Relational algebra . . . 24

19.2 Monoids and strings . . . 24

19.3 Finite automata . . . 25

**20 Change of Basis and Coordinate Systems** **25**

20.1 Coordinates relative to a basis . . . 25

20.2 Change-of-basis matrix . . . 25

**21 Singular Value Decomposition** **25**

21.1 Motivation . . . 25

21.2 Low-rank approximation . . . 26

**22 Algebra in Machine Learning** **26**

22.1 Feature vectors . . . 26

22.2 Linear models . . . 27

22.3 Embeddings . . . 27

**23 Cryptographic Algebra** **27**

23.1 Modular exponentiation . . . 27

23.2 Euler's theorem . . . 27

23.3 Discrete logarithms . . . . . 28

**24 Algorithmic Proof Patterns** **28**

24.1 Invariant arguments . . . . . 28

24.2 Representation arguments . . . . . 28

24.3 Homomorphism arguments . . . . . 28

**25 Worked Examples** **28**

25.1 Solving a system over $\mathbb{Z}_2$ . . . . . 29

25.2 Counting walks in a graph . . . . . 29

25.3 A simple Markov chain . . . . . 29

**26 Exercises** **30**

**27 Summary of Key Ideas** **31**

**28 Formula Sheet** **31**

**Closing Perspective** **32**

# 1 Purpose and Scope

These notes are designed for a mathematics lecture addressed to computer science students. The aim is not to replace a full abstract algebra or linear algebra textbook, but to build a coherent algebraic toolkit that appears repeatedly in algorithms, data analysis, cryptography, coding theory, computer graphics, databases, machine learning, and theoretical computer science.

The guiding principle is simple:

**Algebra studies structure. Computer science uses structure.**

The word *algebra* will be used broadly. We will study numbers, congruences, matrices, vector spaces, linear maps, eigenvalues, Boolean algebra, finite fields, graphs, and algebraic structures. These topics may look different at first, but they share a common pattern: we define objects, operations, rules, and transformations, then use those rules to compute and reason.

**What makes the course computer-science-oriented?**

The notes emphasize constructive definitions, algorithms, computational complexity, finite structures, and examples where algebra is used to encode, transform, compress, classify, search, or secure information.

## 1.1 Prerequisites

The notes assume basic comfort with equations, functions, sets, and elementary proofs. No previous abstract algebra is required. Some examples use programming language vocabulary, but the mathematical arguments do not depend on a specific language.

## 1.2 Core themes

Several themes will appear throughout the notes:

- **Representation:** how to encode objects as vectors, matrices, residues, bits, or polynomials.
- **Transformation:** how operations change objects while preserving some structure.
- **Invariants:** quantities that remain unchanged and help classify objects.
- **Algorithms:** effective procedures, especially elimination, modular exponentiation, and diagonalization.
- **Finite computation:** how algebra behaves over finite sets, especially modulo $n$ and over finite fields.

# 2 Sets, Relations, and Algebraic Language

## 2.1 Sets and operations

A *set* is a collection of objects. For computer science, sets are not just passive containers: they often represent states, symbols, keys, vertices, records, or possible values.

Given a set $S$, a *binary operation* on $S$ is a function

$$\star : S \times S \to S.$$

This means that for every pair $(a, b) \in S \times S$, the result $a \star b$ is again an element of $S$.

**Closure**

A set $S$ is *closed* under an operation $\star$ if for every $a, b \in S$, the result $a \star b$ belongs to $S$.

Closure is a fundamental computational requirement. If an operation leaves the set, then repeated computation may move outside the intended domain.

**Closure and data types**

The integers $\mathbb{Z}$ are closed under addition and multiplication, but the positive integers are not closed under subtraction. In programming terms, an operation may produce a value that does not belong to the type or domain one started with.

## 2.2 Relations and equivalence

A *relation* on a set $S$ is a subset of $S \times S$. We write $a \sim b$ when $(a, b)$ belongs to the relation.

**Equivalence relation**

A relation $\sim$ on $S$ is an *equivalence relation* if for all $a, b, c \in S$:

1. **Reflexivity:** $a \sim a$.
2. **Symmetry:** if $a \sim b$, then $b \sim a$.
3. **Transitivity:** if $a \sim b$ and $b \sim c$, then $a \sim c$.

Equivalence relations formalize the idea of grouping objects that are the same for a chosen purpose. In computer science, this appears in hashing, quotient structures, automata minimization, type equivalence, graph isomorphism, and modular arithmetic.

**Equivalence class**

The equivalence class of $a \in S$ is

$$[a] = \{x \in S : x \sim a\}.$$

The set of all equivalence classes is called the *quotient set*.

## 2.3 Functions and composition

A function $f : A \to B$ assigns to each $a \in A$ a unique value $f(a) \in B$. If $f : A \to B$ and $g : B \to C$, their composition is

$$(g \circ f)(a) = g(f(a)).$$

Composition is generally not commutative: $g \circ f$ and $f \circ g$ may be different, or one may not even be defined.

**Composition as an algebraic operation**

Programs, database queries, graphics transformations, and neural network layers can all be viewed as functions composed in sequence. Algebra gives language for when transformations can be inverted, simplified, or reordered.

# 3 Integers, Divisibility, and Modular Arithmetic

## 3.1 Divisibility

For integers $a, b \in \mathbb{Z}$, with $a \neq 0$, we say that $a$ divides $b$, written $a \mid b$, if there exists $k \in \mathbb{Z}$ such that

$$b = ak.$$

**Greatest common divisor**

For integers $a$ and $b$, not both zero, the greatest common divisor $\gcd(a, b)$ is the largest positive integer that divides both $a$ and $b$.

## 3.2 Euclidean algorithm

The Euclidean algorithm computes $\gcd(a, b)$ using repeated division with remainder:

$$a = bq + r, \qquad 0 \leq r < b.$$

Then

$$\gcd(a, b) = \gcd(b, r).$$

Repeating this process eventually gives remainder zero. The last nonzero remainder is the gcd.

**Computing a gcd**

Compute $\gcd(252, 105)$:

$$252 = 2 \cdot 105 + 42,$$

$$105 = 2 \cdot 42 + 21,$$

$$42 = 2 \cdot 21 + 0.$$

Therefore $\gcd(252, 105) = 21$.

**Bezout identity**

For integers $a, b$, not both zero, there exist integers $x, y$ such that

$$ax + by = \gcd(a, b).$$

The extended Euclidean algorithm finds such $x, y$. This is essential in modular inverses, cryptography, and algorithms over integers.

## 3.3 Congruence modulo $n$

**Congruence**

Let $n \geq 2$. Integers $a, b$ are congruent modulo $n$, written

$$a \equiv b \pmod{n},$$

if $n \mid (a - b)$.

Congruence modulo $n$ means that $a$ and $b$ have the same remainder after division by $n$.

**Clock arithmetic**

Modulo 12, we have

$$15 \equiv 3 \pmod{12}, \qquad 26 \equiv 2 \pmod{12}.$$

This is the arithmetic of a 12-hour clock.

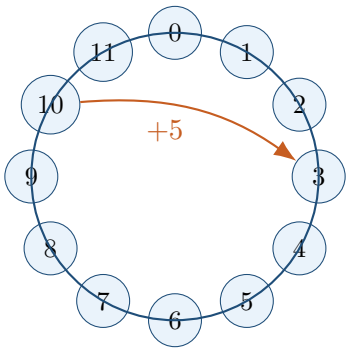



Figure 1: Modulo arithmetic wraps integers around a finite cycle.

## 3.4 Arithmetic in $\mathbb{Z}_n$

The set of residue classes modulo $n$ is

$$\mathbb{Z}_n = \{[0], [1], \ldots, [n-1]\}.$$

Addition and multiplication are defined by

$$[a] + [b] = [a+b], \qquad [a][b] = [ab].$$

**Why this matters in computer science**

Modular arithmetic appears in hashing, pseudo-random number generation, cyclic buffers, checksums, cryptography, finite automata, and low-level machine arithmetic.

## 3.5 Modular inverse

An element $[a] \in \mathbb{Z}_n$ has a multiplicative inverse if there exists $[b] \in \mathbb{Z}_n$ such that

$$[a][b] = [1].$$

Equivalently,

$$ab \equiv 1 \pmod{n}.$$

**Criterion for modular invertibility**

The residue class $[a] \in \mathbb{Z}_n$ is invertible if and only if

$$\gcd(a, n) = 1.$$

**Inverse modulo 26**

Find the inverse of 5 modulo 26. Since

$$26 = 5 \cdot 5 + 1,$$

we get

$$1 = 26 - 5 \cdot 5.$$

Thus

$$-5 \cdot 5 \equiv 1 \pmod{26},$$

so $5^{-1} \equiv -5 \equiv 21 \pmod{26}$.

# 4 Groups: Symmetry and Reversible Computation

## 4.1 Definition of a group

**Group**

A group is a set $G$ with a binary operation $\star$ such that:

1. **Closure:** for all $a, b \in G$, $a \star b \in G$.
2. **Associativity:** $(a \star b) \star c = a \star (b \star c)$.
3. **Identity:** there exists $e \in G$ such that $e \star a = a \star e = a$ for all $a \in G$.
4. **Inverses:** for every $a \in G$, there exists $a^{-1} \in G$ such that $a \star a^{-1} = a^{-1} \star a = e$.

If additionally $a \star b = b \star a$ for all $a, b \in G$, the group is called *abelian*.

Groups model reversible transformations. A transformation is reversible if it can be undone. This is central in permutations, cryptography, error correction, quantum computation, and symmetry.

## 4.2 Examples

- $(\mathbb{Z}, +)$ is an abelian group.
- $(\mathbb{R} \setminus \{0\}, \cdot)$ is an abelian group.
- $(\mathbb{Z}_n, +)$ is an abelian group.
- The set of invertible $n \times n$ matrices over $\mathbb{R}$, with matrix multiplication, is a group denoted $GL_n(\mathbb{R})$. It is nonabelian for $n \geq 2$.
- The set of permutations of $n$ objects is a group denoted $S_n$.

## 4.3 Permutations

A permutation of $\{1, 2, \ldots, n\}$ is a bijection from the set to itself. Permutations compose as functions.

**A permutation**

Let

$$\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 1 & 4 & 2 \end{pmatrix}.$$

This means $\sigma(1) = 3$, $\sigma(2) = 1$, $\sigma(3) = 4$, and $\sigma(4) = 2$.

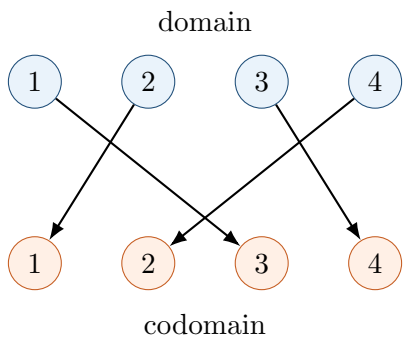



Figure 2: A permutation is a reversible rearrangement.

### 4.4 Subgroups

**Subgroup**

A subset $H \subseteq G$ is a subgroup of a group $G$ if $H$, with the operation inherited from $G$, is itself a group.

For a finite group, a nonempty subset $H \subseteq G$ is a subgroup if it is closed under the operation and closed under inverses.

### 4.5 Cyclic groups

A group $G$ is cyclic if there exists $g \in G$ such that every element of $G$ can be written as a power of $g$. Then $g$ is called a generator.

**Cyclic group modulo 7**

The nonzero residues modulo 7 form a multiplicative group

$$\mathbb{Z}_7^\times = \{1, 2, 3, 4, 5, 6\}.$$

The powers of 3 are

$$3^1 \equiv 3, \quad 3^2 \equiv 2, \quad 3^3 \equiv 6, \quad 3^4 \equiv 4, \quad 3^5 \equiv 5, \quad 3^6 \equiv 1 \pmod 7.$$

Thus 3 generates $\mathbb{Z}_7^\times$.

## 5 Rings and Fields

### 5.1 Rings

**Ring**

A ring is a set $R$ with two operations, addition and multiplication, such that:

1. $(R, +)$ is an abelian group.
2. Multiplication is associative.
3. Multiplication distributes over addition:

$$a(b + c) = ab + ac, \qquad (a + b)c = ac + bc.$$

Many algebraic systems used in computation are rings: integers, polynomials, matrices, residue classes modulo $n$, and formal power series.

### 5.2 Fields

**Field**

A field is a commutative ring with multiplicative identity $1 \neq 0$ in which every nonzero element has a multiplicative inverse.

Examples include $\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$, and $\mathbb{Z}_p$ when $p$ is prime.

**Prime modulus gives a field**

The set $\mathbb{Z}_p$ is a field if and only if $p$ is prime.

### 5.3 Why fields matter

Linear algebra works best over fields because nonzero scalars can be divided. Solving linear systems, defining dimensions, computing bases, and inverting matrices require this property.

In computer science, finite fields are used in:

- error-correcting codes,
- cryptographic protocols,
- hashing and fingerprinting,
- symbolic computation,
- polynomial identity testing.

## 6 Polynomials and Finite Fields

### 6.1 Polynomial rings

Let $F$ be a field. The set $F[x]$ consists of polynomials

$$a_0 + a_1x + \cdots + a_nx^n,$$

where $a_i \in F$. This set is a ring under polynomial addition and multiplication.

**Degree**

The degree of a nonzero polynomial is the largest exponent with nonzero coefficient.

### 6.2 Division algorithm for polynomials

If $f(x), g(x) \in F[x]$, with $g(x) \neq 0$, then there exist unique polynomials $q(x), r(x) \in F[x]$ such that

$$f(x) = q(x)g(x) + r(x), \qquad \deg r < \deg g.$$

This makes polynomial arithmetic resemble integer arithmetic. We can define gcds of polynomials and use Euclidean algorithms.

### 6.3 Finite fields via polynomials

Finite fields of size $p^k$ can be constructed by taking polynomials over $\mathbb{Z}_p$ modulo an irreducible polynomial of degree $k$.

**A field with four elements**

Over $\mathbb{Z}_2$, the polynomial $x^2 + x + 1$ is irreducible. Define

$$\mathbb{F}_4 = \mathbb{Z}_2[x]/(x^2 + x + 1).$$

In this field, the relation

$$x^2 + x + 1 = 0$$

means

$$x^2 = x + 1$$

because over $\mathbb{Z}_2$, subtraction equals addition.

**Computer science perspective**

In finite fields, every operation can be implemented exactly. There is no floating-point roundoff. This is one reason finite fields are essential in exact symbolic computation and coding theory.

## 7 Vectors and Vector Spaces

### 7.1 Vectors as data

In computer science, vectors represent feature lists, images, word embeddings, signals, states, probabilities, graph attributes, and many other structured data objects. Algebra turns such lists into mathematical objects that can be added, scaled, compared, projected, and transformed.

A vector in $\mathbb{R}^n$ is an ordered list

$$v = (v_1, v_2, \ldots, v_n).$$

Addition and scalar multiplication are defined coordinatewise:

$$u + v = (u_1 + v_1, \ldots, u_n + v_n),$$

$$\alpha v = (\alpha v_1, \ldots, \alpha v_n).$$

**Vector space**

A vector space over a field $F$ is a set $V$ whose elements are called vectors, together with vector addition and scalar multiplication, satisfying the usual algebraic rules: associativity, commutativity of addition, zero vector, additive inverses, distributivity, and compatibility with field multiplication.

## 7.2 Linear combinations and span

A linear combination of vectors $v_1, \ldots, v_k$ is an expression

$$a_1 v_1 + \cdots + a_k v_k,$$

where $a_1, \ldots, a_k \in F$.

**Span**

The span of $v_1, \ldots, v_k$ is the set of all their linear combinations:

$$\operatorname{span}(v_1, \ldots, v_k) = \{a_1 v_1 + \cdots + a_k v_k : a_i \in F\}.$$

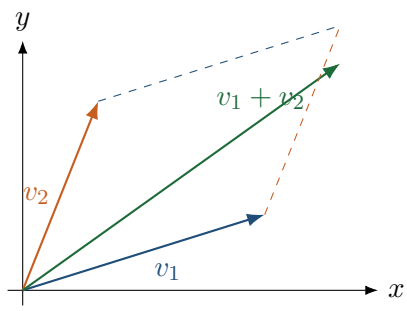



Figure 3: Vector addition and linear combination in the plane.

## 7.3 Linear independence

**Linear independence**

Vectors $v_1, \ldots, v_k$ are linearly independent if

$$a_1 v_1 + \cdots + a_k v_k = 0$$

implies

$$a_1 = a_2 = \cdots = a_k = 0.$$

If a nontrivial combination gives zero, then the vectors are linearly dependent. Dependence means one vector carries information already contained in the others.

**Dependence as redundancy**

In $\mathbb{R}^3$, let

$$v_1 = (1, 0, 1), \quad v_2 = (0, 1, 1), \quad v_3 = (1, 1, 2).$$

Then $v_3 = v_1 + v_2$, so the set is linearly dependent. The third vector does not add a new direction.

### 7.4 Basis and dimension

**Basis**

A basis of a vector space $V$ is a linearly independent set that spans $V$.

Every vector in $V$ has a unique coordinate representation relative to a basis. The number of vectors in any basis of a finite-dimensional vector space is called the dimension.

**Coordinates are representation-dependent**

A vector is a geometric or algebraic object. Its coordinate list depends on the chosen basis. Changing basis changes the representation but not the underlying vector.

## 8 Matrices and Linear Transformations

### 8.1 Matrices

An $m \times n$ matrix over a field $F$ is a rectangular array

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}.$$

A matrix acts on vectors by multiplication. If $A$ is $m \times n$ and $x \in F^n$, then $Ax \in F^m$.

### 8.2 Linear transformations

**Linear map**

A function $T : V \to W$ between vector spaces is linear if for all $u, v \in V$ and scalars $\alpha, \beta$,

$$T(\alpha u + \beta v) = \alpha T(u) + \beta T(v).$$

Every matrix defines a linear map. Conversely, every linear map between finite-dimensional vector spaces can be represented by a matrix once bases are chosen.

### 8.3 Geometric examples

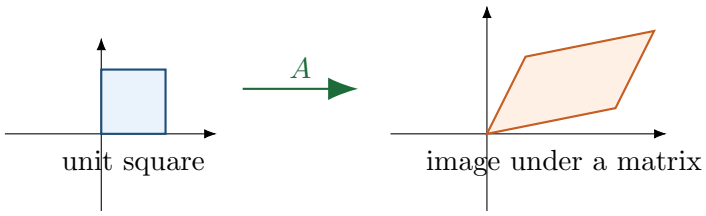



Figure 4: A matrix maps the unit square to a parallelogram.

Typical $2 \times 2$ matrices represent:

- scaling,
- rotation,
- reflection,
- shear,
- projection,
- combinations of these operations.

**Rotation matrix**

A counterclockwise rotation by angle $\theta$ in $\mathbb{R}^2$ is represented by

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}.$$

### 8.4 Matrix multiplication as composition

If $A$ represents $T : U \to V$ and $B$ represents $S : V \to W$, then $BA$ represents the composition $S \circ T$.

The order matters:

$$BAx = B(Ax).$$

First apply $A$, then $B$.

**Pipeline interpretation**

In graphics, data processing, or neural networks, matrix multiplication represents a pipeline of linear transformations. The product matrix compresses several stages into one transformation.

## 9 Systems of Linear Equations

### 9.1 Matrix form

A system of linear equations can be written as

$$Ax = b.$$

Here $A$ is the coefficient matrix, $x$ is the vector of unknowns, and $b$ is the right-hand side.

For example,

$$\begin{cases} 2x + y - z = 1, \\ -x + 3y + 2z = 12, \\ 3x - y + z = 0 \end{cases}$$

becomes

$$\begin{pmatrix} 2 & 1 & -1 \\ -1 & 3 & 2 \\ 3 & -1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 1 \\ 12 \\ 0 \end{pmatrix}.$$

## 9.2 Gaussian elimination

Gaussian elimination uses elementary row operations:

1. swap two rows,
2. multiply a row by a nonzero scalar,
3. add a multiple of one row to another row.

These operations preserve the solution set.

**Row reduction**

Consider

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 2 & 5 & 1 & 9 \\ -1 & -1 & 2 & 0 \end{array}\right].$$

Elimination transforms the augmented matrix into row echelon form. Back-substitution then gives the solution.

## 9.3 Rank

**Rank**

The rank of a matrix is the dimension of the span of its columns. Equivalently, it is the number of pivot columns in its row echelon form.

Rank measures how many independent directions a matrix preserves or produces.

**Rank-nullity theorem**

For a linear map $T : V \to W$, where $V$ is finite-dimensional,

$$\dim V = \dim \ker T + \dim \operatorname{im} T.$$

For an $m \times n$ matrix $A$, this becomes

$$n = \operatorname{nullity}(A) + \operatorname{rank}(A).$$

## 9.4 Computational complexity

Standard Gaussian elimination for an $n \times n$ system has time complexity approximately

$$\mathcal{O}(n^3).$$

This matters for large-scale scientific computing, data analysis, and machine learning. Special matrix structure can reduce cost: sparse matrices, triangular matrices, diagonal matrices, Toeplitz matrices, and low-rank matrices allow more efficient algorithms.

# 10 Determinants and Invertibility

## 10.1 Meaning of determinant

The determinant assigns a scalar to a square matrix. Geometrically, it measures signed volume scaling.

For

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix},$$

we have

$$\det(A) = ad - bc.$$

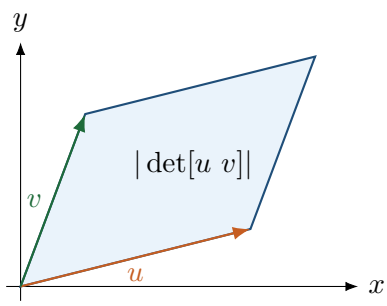



Figure 5: The determinant is the signed area scaling factor in two dimensions.

## 10.2 Invertibility

**Equivalent criteria for invertibility**

For an $n \times n$ matrix $A$ over a field, the following are equivalent:

1. $A$ is invertible.
2. $Ax = 0$ has only the zero solution.
3. The columns of $A$ are linearly independent.
4. The columns of $A$ form a basis of $F^n$.
5. $\operatorname{rank}(A) = n$.
6. $\det(A) \neq 0$.

This theorem is one of the central bridges between algebra, geometry, and computation.

## 10.3 Conditioning

Invertibility is exact algebra. Numerical stability is another issue. A matrix may be invertible but nearly singular, making computations sensitive to small errors.

**Exact versus floating-point algebra**

Over finite fields and rational numbers, we can often do exact algebra. Over floating-point numbers, algorithms must also consider roundoff error and conditioning.

# 11 Inner Products and Orthogonality

## 11.1 Dot product

For vectors $u, v \in \mathbb{R}^n$, the dot product is

$$u \cdot v = u_1 v_1 + \cdots + u_n v_n.$$

The norm is

$$\|v\| = \sqrt{v \cdot v}.$$

## 11.2 Orthogonality

Vectors $u$ and $v$ are orthogonal if

$$u \cdot v = 0.$$

Orthogonality expresses independence in a geometric sense. In data science and algorithms, orthogonal vectors often represent uncorrelated directions, independent features, or separated subspaces.

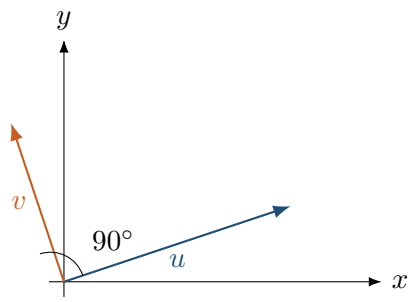



Figure 6: Orthogonal vectors have zero dot product.

## 11.3 Projection

The projection of $v$ onto a nonzero vector $u$ is

$$\mathrm{proj}_u(v) = \frac{v \cdot u}{u \cdot u} u.$$

Projection is the best approximation to $v$ in the one-dimensional subspace spanned by $u$.

**Least-squares intuition**

Least-squares problems find the vector in a subspace closest to a target vector. Algebraically, this is an orthogonal projection problem.

### 11.4 Orthogonal matrices

A square matrix $Q$ is orthogonal if

$$Q^T Q = I.$$

Orthogonal transformations preserve lengths and angles. They are numerically stable and important in QR decomposition, rotations, computer graphics, and machine learning.

## 12 Eigenvalues, Eigenvectors, and Diagonalization

### 12.1 Eigenvectors

**Eigenvalue and eigenvector**

Let $A$ be an $n \times n$ matrix. A nonzero vector $v$ is an eigenvector of $A$ with eigenvalue $\lambda$ if

$$Av = \lambda v.$$

An eigenvector is a direction preserved by the transformation. The vector may be stretched, compressed, or reversed, but it does not change direction.

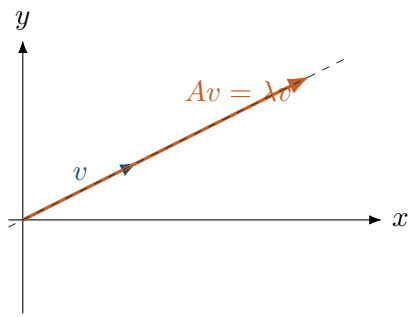



Figure 7: An eigenvector keeps its direction under a linear transformation.

### 12.2 Characteristic polynomial

Eigenvalues are found from

$$\det(A - \lambda I) = 0.$$

The polynomial $p_A(\lambda) = \det(A - \lambda I)$ is called the characteristic polynomial.

**A diagonal matrix**

For

$$D = \begin{pmatrix} 2 & 0 \\ 0 & 5 \end{pmatrix},$$

we have

$$D \begin{pmatrix} 1 \\ 0 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad D \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 5 \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Thus the coordinate axes are eigenvector directions.

### 12.3 Diagonalization

A matrix $A$ is diagonalizable if there exists an invertible matrix $P$ and a diagonal matrix $D$ such that

$$A = PDP^{-1}.$$

The columns of $P$ are eigenvectors, and the diagonal entries of $D$ are eigenvalues.

Diagonalization is useful because powers become easy:

$$A^k = PD^kP^{-1}.$$

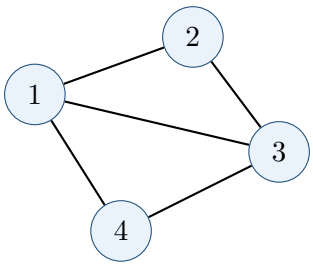
This is important in recurrence relations, Markov chains, graph algorithms, and dynamical systems.

## 13 Graphs and Matrices

### 13.1 Adjacency matrices

A finite graph with vertices $1, \ldots, n$ can be represented by an adjacency matrix $A$, where

$$A_{ij} = \begin{cases} 1, & \text{if there is an edge from } i \text{ to } j, \\ 0, & \text{otherwise.} \end{cases}$$




$$A = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$

Figure 8: A graph and its adjacency matrix.

### 13.2 Walks and powers of adjacency matrices

For an adjacency matrix $A$, the entry $(A^k)_{ij}$ counts the number of walks of length $k$ from vertex $i$ to vertex $j$.

**Algebraic graph counting**

If $A$ is the adjacency matrix of a graph, then the entries of $A^k$ count walks of length $k$. This converts a combinatorial problem into matrix multiplication.

### 13.3 Graph Laplacian

For an undirected graph, the degree matrix $D$ is diagonal with $D_{ii} = \deg(i)$. The graph Laplacian is

$$L = D - A.$$

The Laplacian encodes connectivity and is central in spectral graph theory, clustering, network analysis, and graph-based machine learning.

# 14 Boolean Algebra and Logic

## 14.1 Boolean values

Boolean algebra uses values 0 and 1, representing false and true. The basic operations are:

- AND: $a \wedge b$,
- OR: $a \vee b$,
- NOT: $\neg a$.

| $a$ | $b$ | $a \wedge b$ | $a \vee b$ | $a \oplus b$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 |

Table 1: Truth tables for basic Boolean operations.

## 14.2 Boolean algebra as algebra

Boolean algebra satisfies laws similar to algebraic manipulation:

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c),$$

$$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c),$$

$$a \vee \neg a = 1, \qquad a \wedge \neg a = 0.$$

## 14.3 Boolean rings

If we identify XOR with addition and AND with multiplication, Boolean algebra becomes algebra over $\mathbb{Z}_2$:

$$a \oplus b = a + b \pmod 2, \qquad a \wedge b = ab.$$

This connection is important in circuit design, cryptography, coding theory, SAT solving, and binary linear algebra.

**XOR as addition modulo 2**

The equation

$$x \oplus y \oplus z = 1$$

is equivalent to

$$x + y + z \equiv 1 \pmod 2.$$

Thus systems of XOR constraints can be solved by Gaussian elimination over $\mathbb{Z}_2$.

# 15 Algebra over $\mathbb{Z}_2$: Bits, Codes, and Parity

## 15.1 Vector spaces over $\mathbb{Z}_2$

The set $\mathbb{Z}_2^n$ consists of binary vectors of length $n$. Addition is bitwise XOR.

For example,

$$(1,0,1,1)+(0,1,1,0)=(1,1,0,1)$$

over $\mathbb{Z}_2$.

### 15.2 Parity checks

A parity check is a linear equation over $\mathbb{Z}_2$. For instance,

$$x_1+x_2+x_3+x_4=0$$

means that the number of ones among the bits is even.

**Simple error detection**

A message $(x_1,x_2,x_3)$ can be extended with a parity bit $p$ such that

$$x_1+x_2+x_3+p=0 \pmod 2.$$

If a single bit flips, the parity check fails and an error is detected.

### 15.3 Linear codes

A linear code over $\mathbb{Z}_2$ is a subspace $C\subseteq\mathbb{Z}_2^n$. Its elements are codewords.

Linear codes are described by generator matrices and parity-check matrices. If $G$ is a generator matrix, codewords have the form

$$c=mG,$$

where $m$ is a message vector. If $H$ is a parity-check matrix, valid codewords satisfy

$$Hc^T=0.$$

## 16 Affine Geometry and Homogeneous Coordinates

### 16.1 Affine maps

A linear map sends zero to zero. Many transformations in computer graphics, such as translations, are not linear. They are affine:

$$T(x)=Ax+b.$$

### 16.2 Homogeneous coordinates

Homogeneous coordinates allow affine transformations to be represented by matrices. In two dimensions, write

$$\begin{pmatrix}x\\y\end{pmatrix}\longmapsto\begin{pmatrix}x\\y\\1\end{pmatrix}.$$

Then an affine transformation becomes

$$\begin{pmatrix}A&b\\0&1\end{pmatrix}\begin{pmatrix}x\\y\\1\end{pmatrix}.$$

**Translation matrix**

Translation by $(t_x, t_y)$ is represented by

$$\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}.$$

This is linear in homogeneous coordinates.

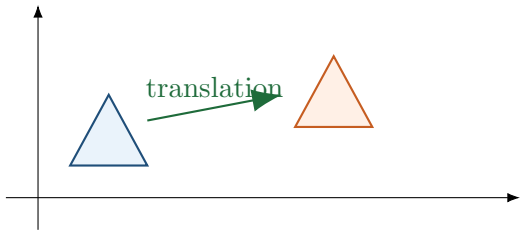



Figure 9: Affine transformations include translations.

# 17 Recurrences, Matrices, and Algorithms

## 17.1 Linear recurrences

A sequence can be defined by a recurrence relation. The Fibonacci sequence satisfies

$$F_{n+1} = F_n + F_{n-1}.$$

This can be written as a matrix equation:

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}.$$

Thus

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F_1 \\ F_0 \end{pmatrix}.$$

## 17.2 Fast exponentiation

Matrix powers can be computed by repeated squaring. This reduces the number of matrix multiplications from $\mathcal{O}(n)$ to $\mathcal{O}(\log n)$.

**Algorithmic lesson**

Algebraic reformulation often leads directly to a faster algorithm. A recurrence becomes a matrix power, and a matrix power can be computed by exponentiation by squaring.

# 18 Semirings and Shortest Paths

### 18.1 Why semirings?

Some algorithms use operations that behave like addition and multiplication, but are not ordinary $+$ and $\cdot$. A semiring formalizes this.

**Semiring**

A semiring is a set $S$ with two operations $\oplus$ and $\otimes$, where $\oplus$ is associative and commutative, $\otimes$ is associative, $\otimes$ distributes over $\oplus$, and there are suitable identity elements.

### 18.2 Examples

- Ordinary arithmetic: $(\mathbb{R}, +, \cdot)$.
- Boolean semiring: $(\{0, 1\}, \vee, \wedge)$.
- Tropical semiring: $(\mathbb{R} \cup \{\infty\}, \min, +)$.

In the tropical semiring, matrix multiplication becomes

$$(AB)_{ij} = \min_k (A_{ik} + B_{kj}).$$

This is exactly the algebraic structure behind shortest path computations.

**Shortest path as algebra**

If $A_{ij}$ stores the length of an edge from $i$ to $j$, then tropical matrix powers encode shortest walks with bounded numbers of edges.

## 19 Algebraic Thinking in Databases and Formal Languages

### 19.1 Relational algebra

Databases use algebraic operations on relations: selection, projection, join, union, difference, and renaming. These operations form the theoretical basis of query languages.

A relation is a set of tuples. A query is a composition of operations that transforms relations into new relations.

**Queries as algebraic expressions**

A database optimizer can rewrite a query expression into an equivalent but faster one. This is algebraic simplification applied to data processing.

### 19.2 Monoids and strings

A monoid is a set with an associative operation and an identity element.

**Monoid**

A monoid is a set $M$ with an operation $\star$ such that:

1. $(a \star b) \star c = a \star (b \star c)$,
2. there exists $e \in M$ with $e \star a = a \star e = a$.

Strings over an alphabet form a monoid under concatenation. The identity is the empty string.

### 19.3 Finite automata

A deterministic finite automaton processes a string symbol by symbol. Each symbol acts as a function on the finite set of states. Thus each word corresponds to a composition of state transformations.

This gives an algebraic view of formal languages: words generate transformations, and languages can be studied through monoids.

## 20 Change of Basis and Coordinate Systems

### 20.1 Coordinates relative to a basis

Let $B = (b_1, \ldots, b_n)$ be a basis of $V$. Every vector $v \in V$ can be written uniquely as

$$v = c_1 b_1 + \cdots + c_n b_n.$$

The coordinate vector of $v$ in basis $B$ is

$$[v]_B = \begin{pmatrix} c_1 \\ \vdots \\ c_n \end{pmatrix}.$$

### 20.2 Change-of-basis matrix

Suppose $B$ and $C$ are two bases. The change-of-basis matrix converts coordinates from one basis to another.

If $P_B$ is the matrix whose columns are the basis vectors of $B$ in standard coordinates, then

$$v = P_B [v]_B.$$

Thus

$$[v]_B = P_B^{-1} v.$$

**Why this matters**

Principal component analysis, Fourier transforms, wavelet transforms, graphics coordinate systems, and embeddings can all be interpreted as choosing a more useful basis.

## 21 Singular Value Decomposition

### 21.1 Motivation

Not every matrix is square, and not every square matrix is diagonalizable. The singular value decomposition works for every real matrix.

**Singular value decomposition**

For every real $m \times n$ matrix $A$, there exist orthogonal matrices $U$ and $V$, and a diagonal rectangular matrix $\Sigma$, such that

$$A = U\Sigma V^T.$$

The diagonal entries of $\Sigma$ are the singular values of $A$. They measure how strongly the matrix stretches along orthogonal directions.

### 21.2 Low-rank approximation

If the singular values are

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0,$$

then keeping only the largest $k$ singular values gives the best rank-$k$ approximation to $A$ in important norms.

**Compression idea**

An image can be stored as a matrix of pixel intensities. A low-rank approximation can keep the main structure while reducing storage. This is algebraic compression.

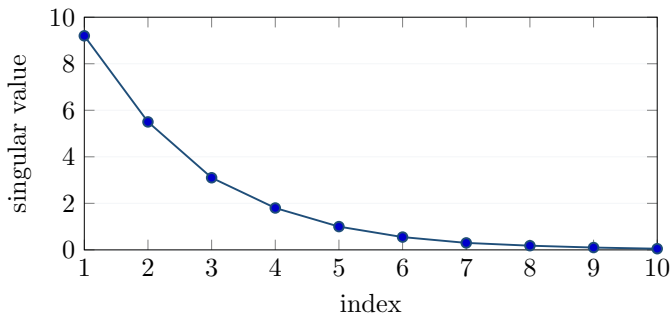



Figure 10: Fast decay of singular values suggests effective low-rank approximation.

## 22 Algebra in Machine Learning

### 22.1 Feature vectors

A data point is often represented as a vector

$$x = (x_1, \ldots, x_n) \in \mathbb{R}^n.$$

A dataset becomes a matrix

$$X = \begin{pmatrix} - & x_1^T & - \\ - & x_2^T & - \\ & \vdots & \\ - & x_m^T & - \end{pmatrix}.$$

Rows represent examples, columns represent features.

### 22.2 Linear models

A linear prediction model has the form

$$\hat{y} = Xw.$$

Training often means choosing $w$ to minimize an error. In least squares, one minimizes

$$\|Xw - y\|^2.$$

When $X^T X$ is invertible, the solution satisfies the normal equations

$$X^T X w = X^T y.$$

### 22.3 Embeddings

Embeddings map discrete objects, such as words, users, items, or nodes, into vector spaces. Once objects are vectors, we can use distances, dot products, projections, and matrix factorization.

**Algebraic viewpoint**

Machine learning often begins by converting objects into vectors. Then linear algebra provides the basic language for similarity, prediction, dimensionality reduction, and optimization.

## 23 Cryptographic Algebra

### 23.1 Modular exponentiation

Modern public-key cryptography uses modular arithmetic. A central operation is

$$a^k \pmod{n}.$$

Efficient computation uses repeated squaring.

**Repeated squaring**

To compute $a^{13}$, write

$$13 = 8 + 4 + 1.$$

Compute $a^2, a^4, a^8$ by repeated squaring, then multiply

$$a^{13} = a^8 a^4 a.$$

Modulo $n$, reduce after every multiplication.

### 23.2 Euler's theorem

**Euler's theorem**

If $\gcd(a, n) = 1$, then

$$a^{\varphi(n)} \equiv 1 \pmod{n},$$

where $\varphi(n)$ is Euler's totient function.

This theorem is one of the algebraic foundations behind RSA-type cryptography.

### 23.3 Discrete logarithms

In a finite cyclic group, exponentiation

$$x = g^k$$

is usually easy to compute. Recovering $k$ from $x$ may be hard for suitable groups. This asymmetry supports discrete-logarithm-based cryptography.

## 24 Algorithmic Proof Patterns

### 24.1 Invariant arguments

An invariant is a property preserved by steps of an algorithm. Algebraic invariants are powerful because they often prove correctness.

**Euclidean algorithm invariant**

Each step replaces $(a, b)$ by $(b, r)$, where $a = bq + r$. The invariant is

$$\gcd(a, b) = \gcd(b, r).$$

Since the invariant is preserved until termination, the final nonzero remainder is the original gcd.

### 24.2 Representation arguments

Many algorithms work by changing representation. Examples:

- row-reducing a matrix without changing the solution set,
- diagonalizing a matrix to compute powers,
- changing basis to reveal independent components,
- moving from strings to automaton transformations,
- moving from graph paths to matrix powers.

### 24.3 Homomorphism arguments

A homomorphism preserves structure. If a complex problem can be mapped to a simpler algebraic structure while preserving the relevant operations, the simpler structure can solve the original problem.

**Homomorphism**

A function $f : G \to H$ between groups is a homomorphism if

$$f(ab) = f(a)f(b)$$

for all $a, b \in G$.

## 25 Worked Examples

## 25.1 Solving a system over $\mathbb{Z}_2$

Solve

$$\begin{cases} x + y + z = 1, \\ y + z = 0, \\ x + z = 1 \end{cases} \qquad \text{over } \mathbb{Z}_2.$$

The augmented matrix is

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{array}\right].$$

Replace row 3 by row 3 plus row 1:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{array}\right].$$

Replace row 2 by row 2 plus row 3:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{array}\right].$$

Thus $z = 0$, $y = 0$, and from the first row $x = 1$. The solution is

$$(x, y, z) = (1, 0, 0).$$

## 25.2 Counting walks in a graph

Let

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$$

be the adjacency matrix of the triangle graph. Then

$$A^2 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}.$$

The entry $(A^2)_{11} = 2$ says that there are two walks of length 2 from vertex 1 back to vertex 1: through vertex 2 or through vertex 3.

## 25.3 A simple Markov chain

A transition matrix

$$P = \begin{pmatrix} 0.8 & 0.3 \\ 0.2 & 0.7 \end{pmatrix}$$

acts on state vectors. If

$$x_0 = \begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

then

$$x_1 = Px_0 = \begin{pmatrix} 0.8 \\ 0.2 \end{pmatrix}.$$

Repeated multiplication models evolution:

$$x_k = P^k x_0.$$

Eigenvectors and eigenvalues help describe long-term behavior.

## 26 Exercises

### A. Modular arithmetic

1. Compute $\gcd(391, 299)$ using the Euclidean algorithm.
2. Find the inverse of 7 modulo 26, if it exists.
3. Solve $5x \equiv 3 \pmod{17}$.
4. List all invertible elements of $\mathbb{Z}_{12}$.

### B. Groups and finite structures

1. Show that $(\mathbb{Z}_n, +)$ is a group.
2. Decide whether $(\mathbb{Z}_n, \cdot)$ is a group. Explain.
3. Write the inverse of the permutation

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 1 & 3 \end{pmatrix}.$$

### C. Linear algebra

1. Determine whether $(1, 2, 3), (2, 4, 6), (0, 1, 1)$ are linearly independent in $\mathbb{R}^3$.
2. Compute the determinant of

$$\begin{pmatrix} 2 & 5 \\ 1 & 3 \end{pmatrix}.$$

3. Find the eigenvalues of

$$\begin{pmatrix} 4 & 0 \\ 0 & -1 \end{pmatrix}.$$

4. Give a matrix representing a shear transformation in $\mathbb{R}^2$.

### D. Algebra over bits

1. Solve over $\mathbb{Z}_2$:

$$x + y = 1, \qquad y + z = 1, \qquad x + z = 0.$$

2. Construct a parity bit for the message $(1, 0, 1, 1, 0)$.
3. Write XOR as addition modulo 2 and simplify $(x \oplus y) \oplus x$.

### E. Graphs and matrices

1. Write the adjacency matrix of a path graph with four vertices.
2. For your matrix $A$, compute $(A^2)_{13}$ and interpret the result.
3. Explain why matrix powers count walks, not necessarily simple paths.

## 27 Summary of Key Ideas

- Modular arithmetic turns infinite integer computation into finite cyclic computation.
- Groups describe reversible operations and symmetries.
- Rings and fields provide algebraic environments where addition and multiplication interact.
- Vector spaces model linear combination, independence, coordinates, and dimension.
- Matrices represent linear transformations and systems of linear equations.
- Determinants, rank, and eigenvalues capture structural properties of matrices.
- Boolean algebra and $\mathbb{Z}_2$-linear algebra are fundamental for logic, circuits, codes, and binary computation.
- Graphs can be studied through matrices; adjacency powers count walks and Laplacians encode connectivity.
- Algebraic reformulation often turns a problem into an efficient algorithm.

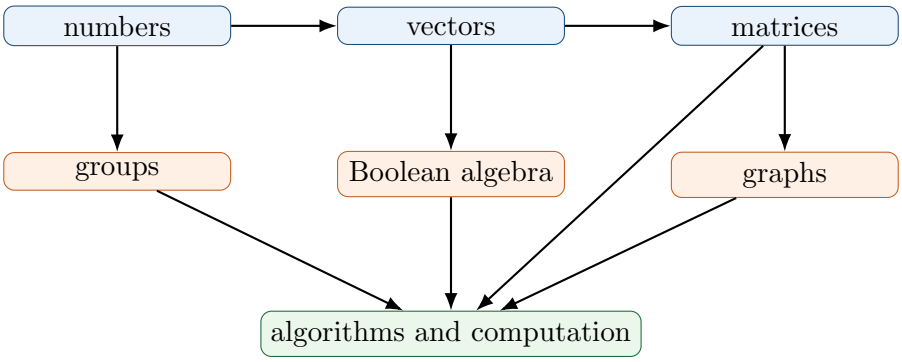



Figure 11: Algebraic concepts form a connected toolkit for computer science.

## 28 Formula Sheet

### Modular arithmetic

$$a \equiv b \pmod{n} \iff n \mid (a-b)$$

$$[a]^{-1} \text{ exists in } \mathbb{Z}_n \iff \gcd(a,n) = 1$$

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

### Linear algebra

$$Ax = b$$

$$\operatorname{rank}(A) + \operatorname{nullity}(A) = n$$

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$$

$$Av = \lambda v$$

$$\det(A - \lambda I) = 0$$

$$A = PDP^{-1}$$

$$A = U\Sigma V^T$$

### Inner products

$$u \cdot v = \sum_i u_i v_i$$

$$\|v\| = \sqrt{v \cdot v}$$

$$\operatorname{proj}_u(v) = \frac{v \cdot u}{u \cdot u} u$$

### Graphs

$$A_{ij} = 1 \iff i \text{ is adjacent to } j$$

$$(A^k)_{ij} = \#\{\text{walks of length } k \text{ from } i \text{ to } j\}$$

$$L = D - A$$

## Closing Perspective

Algebra for computer scientists is not only a list of techniques. It is a way of thinking about computation through structure. The same pattern repeats many times: choose the right objects, define operations, identify invariants, represent transformations, and exploit the resulting structure to compute efficiently.

This is why algebra appears simultaneously in low-level bit operations, high-level machine learning, databases, cryptography, graphics, graph algorithms, and theoretical computer science. Different applications use different algebraic systems, but the underlying habit of thought is the same.